ATIVIDADE PRÁTICA 2 (AV2) · SISTEMAS OPERACIONAIS 2026.2

GenResearch-Servidor

Memória, armazenamento, concorrência e desempenho sob cargas de IA generativa

Estudo prático de chamadas de sistema, mmap, paginação, exclusão mútua e filas em Linux

---

EQUIPE · 7 INTEGRANTES

Bruno Saint Clair Silva Oliveira
João Victor Oliveira Moura
Leonardo de Souza Aragão
Yan Victor Araujo do Nascimento
Gabriel Bastos Pimentel
José Cerqueira da Silva Neto
Marcus Antônio Martins Oliveira Júnior

PERGUNTA NORTEADORA E AMBIENTE

“Como memória principal, memória virtual, armazenamento, caches, filas compartilhadas e sincronização afetam o desempenho, a estabilidade e a correção de uma aplicação de IA generativa ou RAG?”

O que medimos

- Locks, page faults e context switch
- Page cache e memória virtual
- Contenção em seções críticas

Fora do escopo

- Qualidade do texto da LLM
- APIs em nuvem

Hardware e kernel

- Intel Core i5 · 12 threads lógicas
- 24 GB DDR5 (23.731 MB visíveis)
- SSD NVMe · ext4
- Linux x86-64 · Ubuntu LTS · [uname -r]
- CONFIG_TASK_IO_ACCOUNTING ativo

IA local (Ollama)

- RTX 3050 · 6 GB GDDR6
- nomic-embed-text · 768 dim.
- LFM2.5-2.6B · Q4_0 (~1,6 GB)
- Python 3.12 · FastAPI · Uvicorn
- NumPy · SQLite

Do request HTTP à resposta gerada

ARQUITETURA

FastAPI + Uvicorn
API e telemetria

→

Fila + workers
pipeline RAG,
cache LRU

→

SQLite
`genresearch.db`

NumPy
`vectors.npy`

Ollama
GPU

INGESTÃO

Chunking
trechos com overlap

→

Embeddings
768 dim., normalizados L2

→

Persistência
SQLite + vectors.npy

CONSULTA

Vetorizar
query → vetor

→

Similaridade
`matrix @ q`

→

Top-k
só eles vão à RAM

→

Contexto
limite de caracteres

→

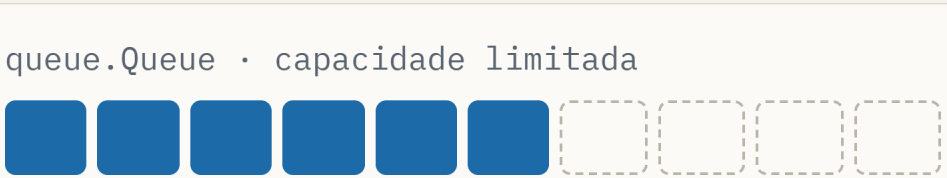
Geração
LFM2.5 na GPU

Fila de produtores e consumidores com cache LRU

PRODUTORES (HTTP)

POST /tasks/query

→ **202 Accepted**



CONSUMIDORES

1, 2 ou 4 workers

not_empty.wait()

Sem busy-waiting

Fila vazia: o worker dorme na variável de condição e não gasta CPU.

Cache LRU

OrderedDict sob threading.Lock; chave SHA-256 da pergunta + parâmetros.

Backpressure

Fila cheia: espera até 3 s; ainda cheia, **HTTP 503**.

PARTE D · 4 THREADS × 2.000 INCREMENTOS = 8.000 ESPERADOS · 3 REPETIÇÕES

Experimento D: condição de corrida versus lock

Como uma atualização se perde

t	Thread A	Thread B	contador
1	lê 41		41
2		lê 41	41
3	grava 42		42
4		grava 42	42 ✗

Dois incrementos, um efeito: *read-modify-write* não é atômico. Largada simultânea via threading.Barrier.

SEM LOCK

≈2.000

de 8.000

≈75% perdidas

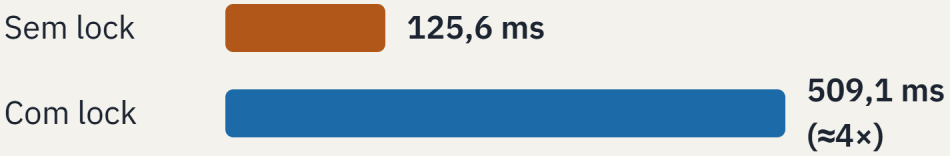
COM LOCK

8.000

nas 3 repetições

0 perdas

Duração média



C1 e C2: leitura convencional versus mmap

C1 · open() + read()

3.500 MB/s

2,79 ms por leitura

Blocos de 64 KB copiados do espaço do kernel para o buffer do usuário.

C2 · mmap()

2.257 MB/s

4,40 ms por leitura

Arquivo projetado na tabela de páginas, sem cópia, paginado sob demanda pela MMU.

> Arquivo de 9,75 MB já no page cache: o custo das faltas de página do mmap superou o da cópia do read().

TELEMETRIA DO KERNEL · /PROC/SELF/IO E GETRUSAGE

Page cache e faltas de página

Por leitura	C1 read()	C2 mmap
rchar	10.221.278 B	158 B
read_bytes	0 B	0 B
Minor faults	0	156
Major faults	0	0

read_bytes = 0: nenhum byte veio do disco; tudo foi servido pelo page cache em RAM.

POR QUE 156 FAULTS?

9,75 MiB ÷ 64 KiB ≈ 156. Cada falta mapeia um bloco de 16 páginas já em cache (*fault-around*), por isso são todas *minor*.

MEMFREE ≠ MEMÓRIA LIVRE

MemFree baixo é normal: o kernel usa RAM ociosa como page cache e a devolve quando preciso. O indicador real é MemAvailable.

CONCORRÊNCIA E CACHE · MÉDIAS DE 3 REPETIÇÕES

Escalabilidade da fila (C3) e eficiência do cache (C4)

C3 · 20 tarefas, fila de capacidade 10

Workers	Vazão	Espera	Speedup
1	58 t/s	115,8 ms	1,00×
2	118 t/s	56,1 ms	2,03×
4	200 t/s	26,1 ms	3,42×

Escala quase linear até 2 workers; com 4, a contenção na fila já aparece (3,42× de 4×).

C4 · cache LRU, 10 consultas

Cache frio

15,15 ms → **0,01 ms**

Cache aquecido

100% de acertos e 99,87% menos tempo por consulta: nenhum acesso redundante ao pipeline.

PRESSÃO DE MEMÓRIA E TOLERÂNCIA A FALHAS

Paginação sob demanda (C5) e resiliência (C6)

C5 · um toque a cada 4 KiB

Alocação	Minor faults	Teórico	RSS pico
100 MB	25.602	25.600	246 MB
250 MB	64.002	64.000	396 MB
500 MB	128.002	128.000	646 MB

Uma falta por página de 4 KiB. Após o GC, RSS volta a 146 MB; 0 major faults, sem swap, sem OOM Killer.

C6 · falhas injetadas, 2 workers

Transitórias
10 de 10 recuperadas com até 2 retentativas

Persistentes
10 descartadas sem travar o pool

Saturação da fila
excedentes rejeitados com HTTP 503

Workers vivos: 2 de 2 e fila consistente nos três cenários.

RELAÇÃO COM SO E LIMITAÇÕES

O que os experimentos dizem sobre IA generativa

C1 · C2

Page cache

Acelera a leitura de índices vetoriais e pesos.

D · C4

Locks

Evitam corrupção de estado; custam latência.

C5

Paginação e RSS

Pesos e contextos ocupam RAM sob demanda; o limite é o OOM Killer.

C3 · C6

Filas e backpressure

Definem a espera e protegem o processo sob sobrecarga.

Limitações

GIL do Python

Sem paralelismo de CPU; mitigado pela carga de I/O e GPU

Métricas por processo

getrusage agrega; por thread exigiria eBPF com root

Nó único

Uma GPU; clusters somam custos de rede e barramento

CONCLUSÃO

A estabilidade e o desempenho de IA generativa dependem do dimensionamento de filas, da contenção de locks e do comportamento da memória virtual.

Artefatos entregues

Relatório técnico, código executável, CSVs de telemetria e gráficos

Uso transparente de IA

Ferramentas de IA apoiaram texto, formatação e análise preliminar de scripts. Todos os dados, logs e tabelas foram coletados em Linux real, sem números simulados.

Obrigado!

GENRESEARCH-SERVIDOR · AV2 SISTEMAS OPERACIONAIS